UNIVERSIDAD NACIONAL DE LOJA

Facultad de la Energía, las Industrias y los Recursos Naturales No Renovables

CARRERA: COMPUTACIÓN

CICLO: 6TO

INTEGRANTES:

CRISTHIAN DANGHELO VALVERDE VILLAMAGUA(Documentación)

ARIEL ALEXANDER TANDAZO RUIZ (Monitor Python)

ALEXANDER ANTONIO SÁNCHEZ AZUERO(Códigos y Pruebas)

MATEO VALENTIN ROJAS GRANDA(Modelo Formal)

TIPO:

INFORME TÉCNICO ANÁLISIS DE SECUENCIAS DE EVENTOS CON AUTÓMATAS FINITOS

"MONITOR DE TRANSACCIONES DE UNA TIENDA EN LÍNEA MEDIANTE AUTÓMATAS FINITOS"

FECHA: 29/10/2026

MATERIA: TEORÍA DE AUTÓMATAS Y COMPUTABILIDAD AVANZADA

RESPONSABLE: ING. SUING ALBITO GENOVEVA JACKELINNE

1. Introducción

Una tienda en línea registra los eventos ordenados asociados con cada compra: con el inicio, autorización o rechazo del pago, confirmación, cancelación, devolución, reembolso, las bitácoras permiten representar cada sesión como una cadena y comprobar anomalías estructurales mediante lenguajes regulares. El objetivo del proyecto es construir y verificar el monitor de secuencias simuladas que marque una sesión como anómala si aparece el BC consecutivo, CC consecutivo, si comienza con D o si comienza con R, el sistema no procesa pagos reales, no utiliza datos personales que no pretende detectar toda irregularidad de una tienda, únicamente que compruebas estas cuatro reglas, incluye la generación reproducible de datos CSV, el diseño de un AFND por patrón, con su unión mediante las transiciones ε,la eliminación de ε, etc, se va a evaluar el monitor en Python con archivos JFLAP.

2. Modelo formal

- **Definición del alfabeto**

  El alfabeto planteado se encuentra conformado por ocho símbolos los cuales son:

  $\sum = \{B, P, F, C, X, D, R, V\}$

  Cada uno de estos símbolos representan una acción de una sesión de compra:

Simbolo	**Evento**	**Relevancia**
B	Inicio de compra	Inicio del flujo de pedido
P	Pago aprobado	Registra aprobación de pago
F	Pago rechazado	Registra rechazo de pago
C	Compra confirmada	Confirma el pedido
X	Compra cancelada	Cancela el pedido
D	Devolución registrada	Registre devolucion
R	Reembolso realizado	Registra reembolso
V	Consulta de producto	No activa un patrón que puede interrumpir consecutividad

Tabla.1: Tabla del Alfabeto utilizado

Patron/ Definicion	**Pertenecen**	**No pertenecen**
P1: Contieen BC consecutivo L_1 = {w Σ* | x,y Σ*: w = xBCy]	BC, VBC, BPBC	BPC, BVC, CB
P2. Contiene CC consecutivoL_2 = {w Σ* | x,y Σ*: w = xCCy}	CC, BPCC, VCC	BPC, CVC, CPC
P3. Empieza con DL_3 = {w Σ* | y Σ*: w = Dy}	DPC, DVB, DBPC	BPCD, VDB, BDR
P4. Empieza con RL_4 = {w Σ* | y Σ*: w = Ry}	RPC, RVB, RBPC	BPCDR, VRB, BPR

Tabla.2 Tabla de patrones y lenguaje global

El lenguaje global de anomalías es L_anom = L_1 L_2 L_3 L_4 con la clasificación binaria se define como anómala si w ∪∈ y L_normal en caso contrario, un misma sesión podría satisfacer varios patrones; el monitor informa todos los identificados.

- **Justificacion y delimitacion:**

El P1 detecta una confirmación al cual actúa de forma inmediata tras un inicio pero ningún evento intermedio, es una aproximación conservadora a la regla comercial más amplia con la compra confirmada sin pago aprobado previo en la propuesta, la longitud de la secuencia elemental de cada patrón es de uno o dos símbolos (4) y P1/P2 son patrones de tipo ≤«contiene» con un diseño natural AFND.

Cadena	Pratones	Resultados
BPC	–	normal
BVC	–	normal
BC	P1	anomala
BPCC	P2	anomala
DVB	P3	anomala
RBC	P1 y P4	anomala
BFC	–	normal

Tabla 3. Tabla de Cadenas

3. Conjunto de datos

Para el origen de los datos se optó por datos sintéticos: datos/generador.py hace uso de random.Random(2026) para la validación y random.Random(2027) para prueba. A partir de plantillas normales como BPC, BPCDR, BFBPC, BX y BFX y se insertan entre 0 y 3 consultas V. Se completa a un mínimo de tres símbolos y se mantienen secuencias de hasta 20 eventos.

Conjunto	Normales	Anomalas	Casi anomalas*	Total	Eventos
Validación	246	54 (18,00%)	40	300	1.557
Prueba	49	11(18,33%)	8	60	309

Tabla 4: Tabla de estados normales y anomalos.

- **Estructura, etiquetado y ejemplo**

Los CSV registran id_evento, fecha_hora, id_sesion, actor, evento y símbolo además validacion.csv incluye también etiqueta y tipo_patron, en prueba.csv no se tiene estas dos columnas y clave_prueba.csv almacena la respuesta por sesion para la evaluación separada. Las etiquetas se calculan con las etiquetas asignadas por construcción y contrastadas con reglas de cadena de los cuatro lenguajes.

id_evento	id_sesion	evento	símbolo	etiqueta
1	V-0001	Consulta producto	V	normal
2	V-0001	Inicio compra	B	normal
3	V-0001	Pago rechazado	F	normal
4	V-0001	Compra cancelada	X	normal

Tabla 5: Conjunto de datos a eventos

4. Diseño de los AFND

Cada automata cuenta con la quintupla M = $(Q, \Sigma, \delta, q0, F)$, en el cual Q es un conjunto finito de estados, $\delta : Q \times \Sigma \to \mathcal{P}(Q)$ devuevle los estados que son alcanzables, q0 es el inicial y F es el conjunto de aceptacion, dentro de las tablas ⌀ indica la ausencia de transición.

P1- contiene la subcadena BC

$M1$ = ({$a0$, $a1$, $a2$}, Σ, $\delta 1$, $a0$, {$a2$}). a0 sigue buscando B; a1 recuerda una B inmediatamente anterior; a2 indica que ya se encontró BC y permanece aceptando. Es no determinista porque $\delta 1(a0, B)$ = {$a0$, $a1$}.

Estado	B	C	O = {P,F,X,D,R,V}
→ a0	{a0,a1}	{a0}	{a0}
a1	⌀	{a2}	⌀
*a2	{a2}	{a2}	{a2}

Tabla.6: Diseño de tabla del AFND

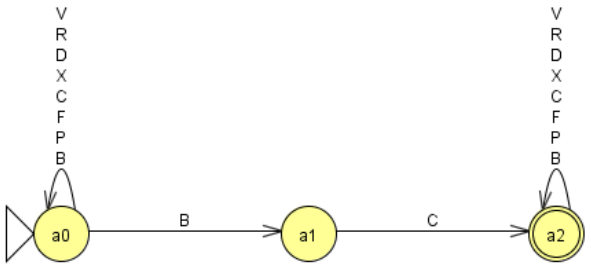


Figura.1 - AFND P1

P2- contiene la subcadena CC

$M2$ = ({$b0$, $b1$, $b2$}, Σ, $\delta 2$, $b0$, {$b2$}). b0 busca la primera C; b1 recuerda que la anterior fue C; b2 ya ha reconocido CC. Es no determinista porque $\delta 2(b0, C)$ = {$b0$, $b1$}.

Estado	**C**	**O = {B,P,F,X,D,R,V}**
→ b0	{b0,b1}	{b0}
b1	{b2}	⌀
*b2	{b2}	{b2}

Tabla.7: Diseño de tabla del AFND P2

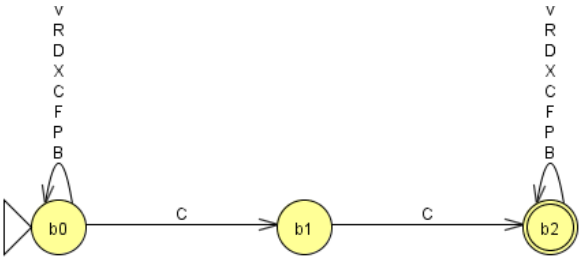


Figura.2 - AFND P2

P3- la sesión comienza con D

$M3$ = ({$c0$, $c1$}, Σ, $\delta 3$, $c0$, {$c1$}). c0 espera el primer símbolo, c1 ha reconocido D inicial. No tiene bifurcación: es determinista parcial, que también puede representarse como AFND porque un AFD es un caso particular de AFND.

Estado	D	O = {B,P,F,C,X,R,V}
→ c0	{c1}	⌀
*c1	{c1}	{c1}

Tabla.8: Diseño de tabla del AFND P3

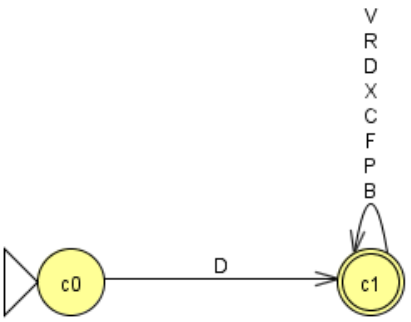


Figura- 3 - AFND P3

P4- la sesión comienza con R

$M4$ = ({$d0$, $d1$}, Σ, $\delta 4$, $d0$, {$d1$}). d0 espera el primer símbolo; d1 ya ha reconocido R inicial. Tampoco hay bifurcación; se incluye en la unión de AFND.

Estado	R	O = {B,P,F,C,X,R,V}
→ d0	{d1}	⌀
*d1	{d1}	{d1}

Tabla.9: Diseño de tabla del AFND P4

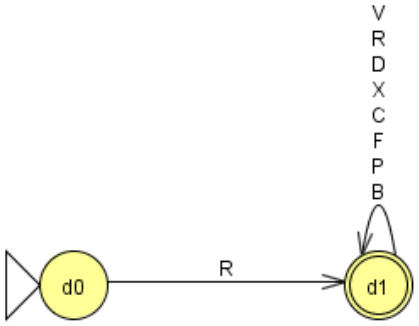


Figura-4 - AFND P4

5. Unión con transiciones ε y su eliminación

La unión que incorpora el nuevo estado inicial s con las transiciones de $\delta(s, \varepsilon)$ = {$a0$, $b0$, $c0$, $d0$}, cada uno de los estados es Q = {s, $a0$, $a1$, $a2$, $b0$, $b1$, $b2$, $c0$, $c1$, $d0$, $d1$} con los estados F = {$a2$, $b2$, $c1$, $d1$}, cada transición ε permite activar uno de los cuatro patrones sin consumir un carácter, vamos a utilizar los siguientes pasos:

Simbolo	**δ'(s, símbolo)**
B	{a0,a1,b0}
P,F,X,V	{a0,b0}
C	{a0,b0,b1}
D	{a0,b0,c1}
R	{a0,b0,d1}

Tabla.10: Tabla de símbolos de Autómata.

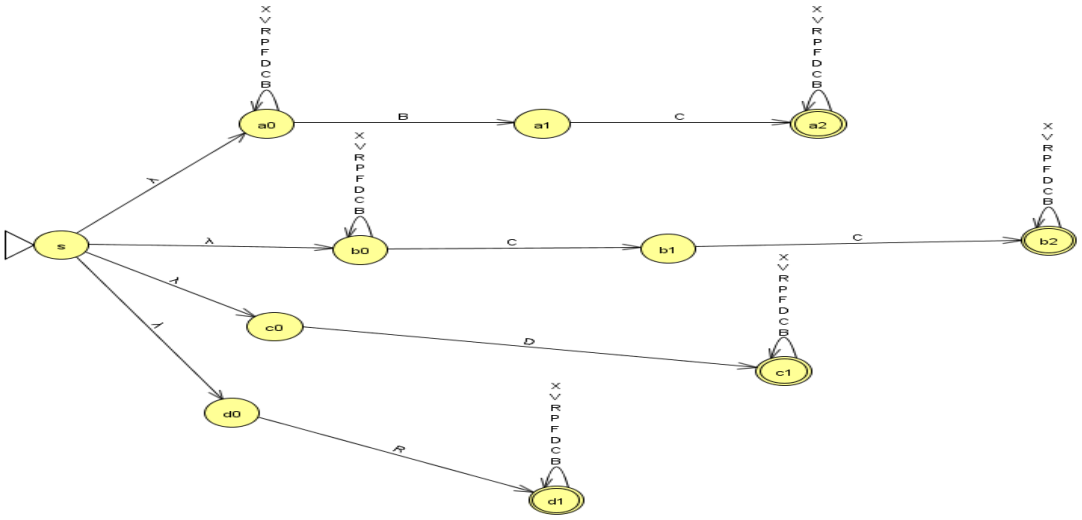


Figura.5 - Unión ε de los cuatro AFND

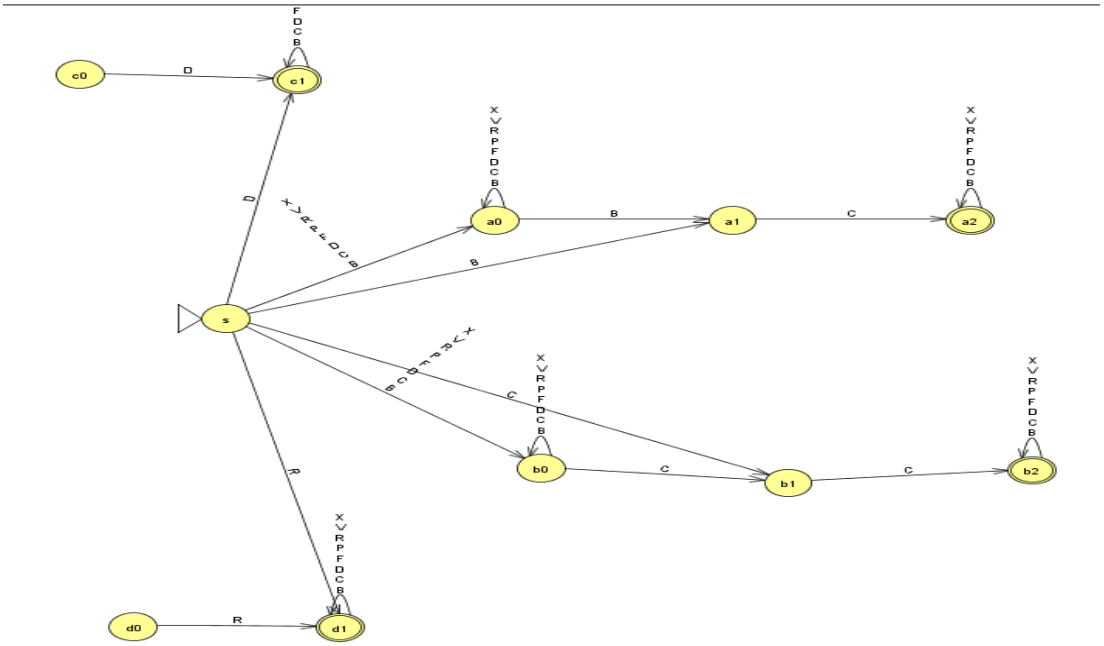


Figura.6 - Eliminación ε de los cuatro AFND

6. Conversión a AFD y minimización

Conversión del AFND a AFD por subconjuntos

Al aplicar la construcción de subconjuntos al AFND sin Ɛ y al conservar únicamente estados que son accesibles desde {S}, se obtienen 37 estados desde A0 hasta A36, un subconjunto es final si este tiene a2,b2,c1 o d1, en la tabla se incluyen todas las transiciones: O={P,F,X,V}

Estado	Subconjunto	B	C	O	D	R
→A0	s	A1	A3	A2	A4	A5
A1	a0,a1,b0	A1	A6	A2	A2	A2
A2	a0,b0	A1	A3	A2	A2	A2
A3	a0,b0,b1	A1	A7	A2	A2	A2
*A4	a0,b0,c1	A8	A9	A4	A4	A4
*A5	a0,b0,d1	A10	A11	A5	A5	A5
*A6	a0,a2,b0,b1	A12	A14	A13	A13	A13
*A7	a0,b0,b1,b2	A15	A7	A16	A16	A16
*A8	a0,a1,b0,c1	A8	A17	A4	A4	A4
*A9	a0,b0,b1,c1	A8	A18	A4	A4	A4
*A10	a0,a1,b0,d1	A10	A19	A5	A5	A5
*A11	a0,b0,b1,d1	A10	A20	A5	A5	A5
*A12	a0,a1,a2,b0	A12	A6	A13	A13	A13
*A13	a0,a2,b0	A12	A6	A13	A13	A13
*A14	a0,a2,b0,b1,b2	A21	A14	A22	A22	A22
*A15	a0,a1,b0,b2	A15	A14	A16	A16	A16
*A16	a0,b0,b2	A15	A7	A16	A16	A16
*A17	a0,a2,b0,b1,c1	A23	A25	A24	A24	A24
*A18	a0,b0,b1,b2,c1	A26	A18	A27	A27	A27
*A19	a0,a2,b0,b1,d1	A28	A30	A29	A29	A29
*A20	a0,b0,b1,b2,d1	A31	A20	A32	A32	A32
*A21	a0,a1,a2,b0,b2	A21	A14	A22	A22	A22
*A22	a0,a2,b0,b2	A21	A14	A22	A22	A22
*A23	a0,a1,a2,b0,c1	A23	A17	A24	A24	A24
*A24	a0,a2,b0,c1	A23	A17	A24	A24	A24
*A25	a0,a2,b0,b1,b2,c1	A33	A25	A34	A34	A34
*A26	a0,a1,b0,b2,c1	A26	A25	A27	A27	A27
*A27	a0,b0,b2,c1	A26	A18	A27	A27	A27
*A28	a0,a1,a2,b0,d1	A28	A19	A29	A29	A29
*A29	a0,a2,b0,d1	A28	A19	A29	A29	A29
*A30	a0,a2,b0,b1,b2,d1	A35	A30	A36	A36	A36
*A31	a0,a1,b0,b2,d1	A31	A30	A32	A32	A32
*A32	a0,b0,b2,d1	A31	A20	A32	A32	A32
*A33	a0,a1,a2,b0,b2,c1	A33	A25	A34	A34	A34
*A34	a0,a2,b0,b2,c1	A33	A25	A34	A34	A34
*A35	a0,a1,a2,b0,b2,d1	A35	A30	A36	A36	A36
*A36	a0,a2,b0,b2,d1	A35	A30	A36	A36	A36

Tabla.11: Conversación AFD

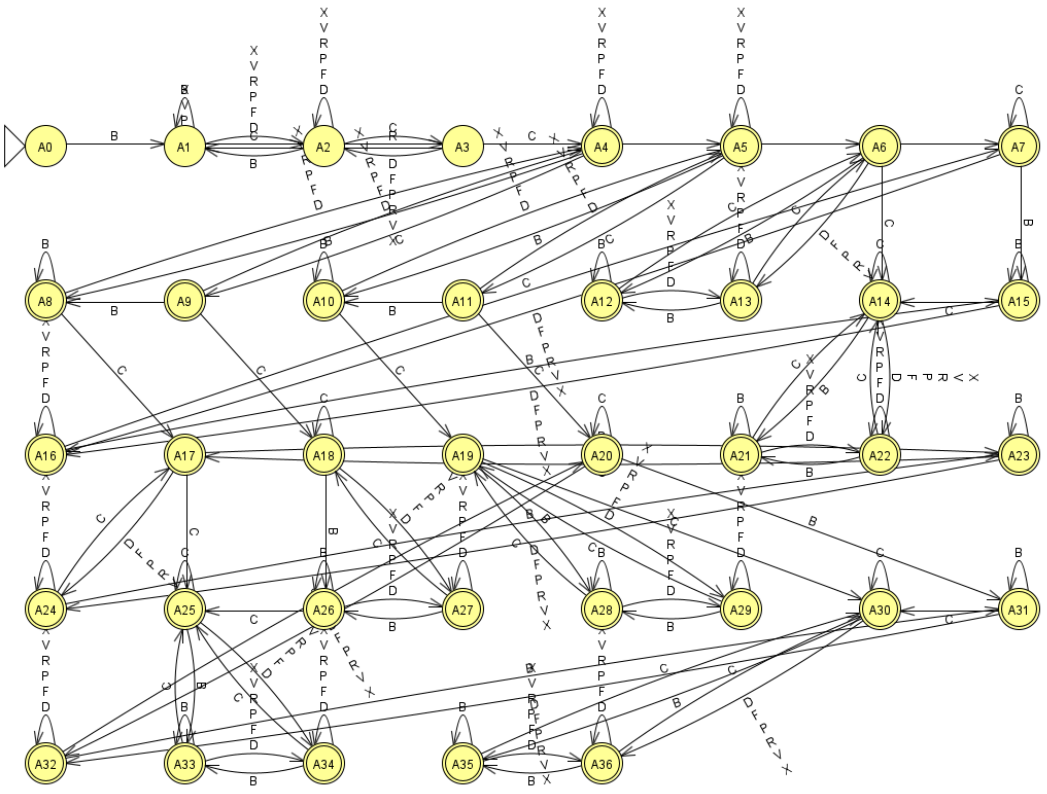


Figura.7 - Autómata de 37 estados

Minimización de AFD y equivalencia

El proceso comienza por separar los estados finales de los no finales, luego se revisan sus transiciones para saber cuales cuentan con el mismo comportamiento y con ello los 37 subconjuntos se pueden reducir a cuatro grupos equivalentes, manteniendo con ello el mismo funcionamiento y logrando reconocer las mismas secuencias.

Mínimo	**Subconjuntos del AFD**	**Interpretación**
M0	A0	Inicio, aun no se consume símbolo
M1	A1, A3	Ultimo simbolo Bo C sin anomalia
M2	A2	Sin anomalia y sin B/C inmediata
*M3	A4-A36	Se ha reconocido al menos un patrón

Tabla.12: Tabla de minimización de AFD

Estado	B	P	F	C	X	D	R	V
→ M0	M1	M2	M2	M1	M2	M3	M3	M2
M1	M1	M2	M2	M3	M2	M2	M2	M2
M2	M1	M2	M2	M1	M2	M2	M2	M2
*M3	M3	M3	M3	M3	M3	M3	M3	M3

Tabla.13: Tabla de estados AFD

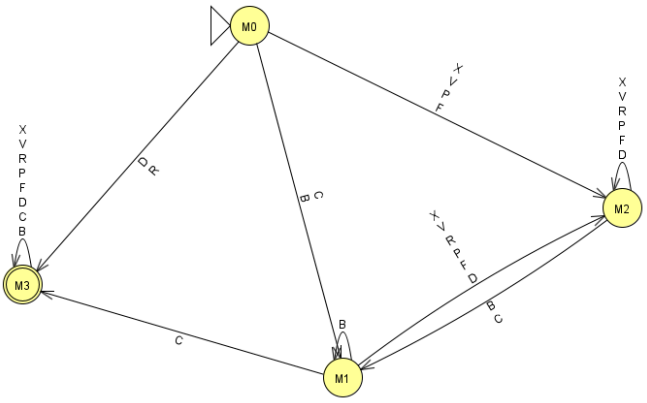


Figura.8 - AFD mínimo de cuatro estados

El estado M3 representa a al menos uno de los patrones anómalos por lo que cuando el autómata llegue a ese estado se quedará atrapado en ese estado sin importar los siguientes símbolos.

Verificación de equivalencia

Para poder comprobar que la minimización es verdaderamente equivalente se utilizó la herramienta compare equivalence que tiene JFLAP y se comparó los dos AFD y se confirmó que son equivalentes ya que permite comprobar que los dos autómatas pueden aceptar y rechazar las mismas cadenas.

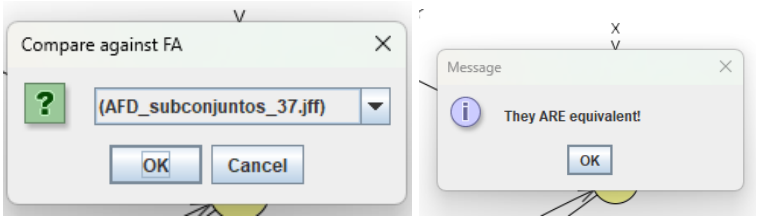


Figura.9 - Verificación de equivalencia

7. Implementación del monitor

La implementación del monitor se organizó en módulos independientes para separar la definición formal del autómata, el procesamiento de los datos y la exposición de resultados en la aplicación web.

7.1 Representación del autómata

El alfabeto utilizado es:

```
ALPHABET = ("B", "P", "F", "C", "X", "D", "R", "V")
```

La tabla de transición se representa mediante un diccionario anidado:

```
TRANSITIONS = {
    "M0": {
        "B": "M1", "P": "M2", "F": "M2", "C": "M1",
        "X": "M2", "D": "M3", "R": "M3", "V": "M2"
    },
    "M1": {
        "B": "M1", "P": "M2", "F": "M2", "C": "M3",
        "X": "M2", "D": "M2", "R": "M2", "V": "M2"
    },
    "M2": {
        "B": "M1", "P": "M2", "F": "M2", "C": "M1",
        "X": "M2", "D": "M2", "R": "M2", "V": "M2"
    },
    "M3": {
        "B": "M3", "P": "M3", "F": "M3", "C": "M3",
        "X": "M3", "D": "M3", "R": "M3", "V": "M3"
    }
}
```

7.2 Simulación de una cadena

La función simulate() inicia el recorrido en M0 y procesa los símbolos uno por uno:

```
def simulate(sequence: str) -> list[dict[str, str]]:
    normalized = validate_sequence(sequence)
    state = INITIAL_STATE
    trace = [{"from": "—", "symbol": "Inicio", "to": state}]

    for symbol in normalized:
        next_state = TRANSITIONS[state][symbol]
        trace.append({
            "from": state,
            "symbol": symbol,
            "to": next_state
        })
        state = next_state

    return trace
```

7.3 Detección de patrones

La detección de los patrones individuales se encuentra en monitor.py. Se implementa mediante las reglas definidas para los cuatro lenguajes anómalos:

```
def detect_patterns(sequence: str) -> list[dict[str, str]]:
    sequence = sequence.upper()

    rules = (
        ("P1", "BC" in sequence),
        ("P2", "CC" in sequence),
        ("P3", sequence.startswith("D")),
        ("P4", sequence.startswith("R")),
    )

    return [
        {
            "id": pattern["id"],
            "name": pattern["name"],
            "rule": pattern["rule"]
        }
        for pattern in PATTERNS
        for pattern_id, matched in rules
        if pattern["id"] == pattern_id and matched
    ]
```

7.4 Análisis completo de una secuencia

La función analyze_sequence() integra la detección de patrones, la simulación del AFD y la construcción de la respuesta:

```
def analyze_sequence(sequence: str) -> dict[str, Any]:
    normalized = sequence.strip().upper()
    detected = detect_patterns(normalized)
    trace = simulate(normalized)

    return {
        "sequence": normalized,
        "valid": True,
        "status": "Anómala" if is_anomalous else "Normal",
        "patterns": detected,
        "trace": trace,
        "final_state": trace[-1]["to"],
        "pattern_evidence": pattern_evidence(
            normalized,
            detected
        ),
    }
```

8. Resultados

El monitor se evaluó con dos conjuntos sintéticos de semilla fija: validación (300 sesiones, 1.557 eventos) y prueba (60 sesiones, 309 eventos). Cada sesión se procesó con el AFD mínimo y se clasificó como anómala si terminó en M3. Luego se comparó con su etiqueta. Las anomalías se distribuyen así en validación: P1 = 14, P2 = 14, P3 = 13 y P4 = 13; en prueba: 3, 3, 3 y 2. Esta distribución es la que asigna el generador, y el patrón concreto lo informa detect_patterns, porque el AFD solo distingue anómala de normal.

Conjunto	Normales	Anómalas	VP	FP	FN	VN	Precisión	Exhaustividad
Validación	246	54 (18 %)	54	0	0	246	100%	100%
Prueba	49	11 (18,33 %)	11	0	0	49	100%	100%

Tabla 14. Métricas de clasificación

Autómata	Estados	Transiciones	Tiempo mediano (ms)
AFND unido con ε	11	58	2,695
AFND sin ε	11	74	1,765
AFD por subconjuntos	37	296	—------
AFD mínimo	4	32	0,074

Tabla 15. Tamaño y tiempo de ejecución

Los 1,765 ms del AFND sin ε respaldan la idea de que eliminar las ε evita calcular la clausura en cada paso.

9. Límites del modelo

Regla que un autómata finito no puede verificar. La tienda tiene una regla de negocio: *el total* acumulado de los reembolsos no debe superar el total acumulado de los pagos aprobados. Un autómata finito tiene un número fijo de estados, y cada estado solo puede recordar una cantidad finita de información. Para evaluar esta regla se necesitaría recordar la diferencia entre lo pagado y lo reembolsado, que puede crecer sin límite. Por esta razón el alfabeto Σ no incluye importes y el monitor no evalúa la regla, que queda fuera del modelo.

Por qué los patrones P1 a P4 sí son regulares. Cada uno requiere recordar una cantidad finita de información: el último símbolo leído (BC y CC) o el primer símbolo de la sesión (D y R). Por eso el AFD mínimo necesita solo 4 estados.

Efecto sobre el monitor. La cadena BPRR tiene dos reembolsos y un solo pago, así que viola la regla. El monitor la clasifica como normal, porque no contiene BC ni CC y no empieza con D ni R. Esto es una consecuencia del límite del modelo, no un error de implementación.

Formalización simplificada. Para analizar el límite se toma una versión de la regla en la que cada P suma una unidad y cada R resta una unidad:

L_regla = { w ∈ Σ* | para todo prefijo u de w, |u|_R ≤ |u|_P }

donde |u|_x es el número de apariciones del símbolo x en u.

10. Conclusiones

Respuesta a la pregunta guía: ¿qué secuencias de eventos de un sistema real conviene validar o vigilar, y cómo puede un autómata finito detectarlas?

Conviene vigilar las secuencias de eventos cuyo orden revela un fallo o un uso indebido, aunque cada evento, considerado de forma aislada, sea legítimo. En la tienda en línea se definieron cuatro: la confirmación inmediata tras el inicio de compra (BC), dos confirmaciones consecutivas (CC), y las sesiones que comienzan con una devolución (D) o con un reembolso (R). Un autómata finito puede detectarlas porque cada sesión se representa como una cadena sobre Σ y cada patrón define un lenguaje regular. El autómata recorre la cadena una sola vez y el estado en que termina indica si la sesión es anómala o normal.

Aprendizajes

Diseño no determinista. P1 y P2 son patrones del tipo la sesión contiene la secuencia... y su diseño natural es un AFND. P3 y P4 son deterministas, y se representaron como AFND para unirlos con transiciones ε.

Límites de la regularidad. Existen reglas, como la de reembolsos y pagos, que ningún autómata finito puede verificar. Identificarlas y dejarlas fuera del modelo forma parte del diseño.

Transformaciones. La unión con transiciones ε produjo un AFND de 11 estados. Su conversión mediante construcción de subconjuntos dio un AFD de 37 estados accesibles, y la minimización lo redujo a 4. La reducción se debe a que, una vez detectado un patrón, el autómata permanece en un estado de aceptación del que no sale

Posibles mejoras

Asignar estados de aceptación distintos a cada patrón para que el monitor informe cuál de los cuatro se detectó, ya que el AFD mínimo actual sólo distingue entre anómala y normal.

Incorporar nuevos patrones, por ejemplo la repetición de pagos rechazados.

Referencias y declaración de uso de IA

Herramientas de IA empleadas, para qué y en qué partes.

Propósito del uso: Las herramientas Claude y ChatGPT se utilizaron como apoyo para comprender los conceptos de teoría de autómatas, aclarar los requisitos de la actividad, revisar la estructura de la propuesta y mejorar la redacción de algunas secciones del informe, además de apoyo al desarrollo del código en Python.

Partes asistidas: La asistencia se centró en la interpretación de las instrucciones de la actividad, la organización y revisión de la sección correspondiente al conjunto de datos, y la comprensión de los procesos de validación, prueba y clasificación de sesiones mediante autómatas, además del código Python.

Prompts utilizados:

«Actúa como un guía en la realización del siguiente trabajo»

«Analiza nuestra propuesta del monitor de transacciones de una tienda en línea junto con la guía de evaluación sumativa. Ayúdame a delimitar el alcance del proyecto, utilizando objetivos que podamos demostrar y evitando afirmaciones demasiado amplias.»

«Explica la unión de los cuatro autómatas mediante transiciones épsilon, su eliminación, la conversión a AFD mediante construcción de subconjuntos y la minimización, mostrando las tablas correspondientes.»

«Prepara los archivos Python necesarios para implementar el monitor, incluyendo el generador de datos sintéticos, la representación de los autómatas, la lectura de archivos CSV, el procesamiento de sesiones y el cálculo de métricas.»

Link de Github:

https://github.com/AlexanderSanchezCS/Monitor-Transiciones-Automatas.git

Referencias en formato IEEE

[1] J. E. Hopcroft, R. Motwani y J. D. Ullman, *Introducción a la teoría de autómatas, lenguajes y computación*, 3.ª ed. Pearson, 2006. [Título original: *Introduction to Automata Theory, Languages, and Computation*].

[2] M. Sipser, *Introducción a la teoría de la computación*, 3.ª ed. Cengage Learning, 2013. [Título original: *Introduction to the Theory of Computation*].

[3] S. H. Rodger y T. W. Finley, *JFLAP: Paquete interactivo de lenguajes formales y autómatas*. Jones & Bartlett Publishers, 2006. [Título original: *JFLAP: An Interactive Formal Languages and Automata Package*].

[4] JFLAP, «JFLAP versión 7.1: Herramienta para el diseño y simulación de autómatas y lenguajes formales». [En línea]. Disponible en: https://www.jflap.org/framebody.html.

[5] Python Software Foundation, «Módulo csv: Lectura y escritura de archivos CSV», *Documentación oficial de Python*. [En línea]. Disponible en: https://docs.python.org/es/3/library/csv.html.

[8] Python Software Foundation, «Módulo time: Acceso y medición del tiempo», *Documentación oficial de Python*. [En línea]. Disponible en: https://docs.python.org/es/3/library/time.html.